

ANÁLISE COMPARATIVA DE ALGORITMOS DE BUSCA APLICADOS AO PROBLEMA LIGHTS OUT

Trabalho Acadêmico

Disciplina: Inteligência Artificial

Instituição: Universidade Tuiuti do Parana

Autor: Nicolas Siqueira Damasio dos Santos

Data: Abril de 2026

RESUMO

Este trabalho apresenta uma análise experimental de diferentes estratégias de busca aplicadas ao desafio computacional conhecido como Lights Out. O estudo implementou e avaliou seis algoritmos distintos: Busca em Largura (BFS), Busca em Profundidade (DFS), Busca Heurística A*, Busca Gulosa, Subida de Encosta (Hill Climbing) e Têmpera Simulada (Simulated Annealing). A metodologia envolveu a criação de um sistema automatizado de testes com tabuleiros de dimensões variadas (3x3, 5x5, 7x7, 10x10 e 100x100) e níveis crescentes de complexidade. Os resultados demonstraram que algoritmos informados, particularmente o A*, apresentaram desempenho superior em termos de taxa de sucesso e eficiência computacional, enquanto métodos de busca local mostraram limitações significativas na resolução do problema. As métricas coletadas incluem tempo de execução, número de movimentos, quantidade de estados explorados e taxa de convergência.

Palavras-chave: Inteligência Artificial; Algoritmos de Busca; Lights Out; Busca Heurística; Otimização Local.

1 INTRODUÇÃO

O problema Lights Out constitui um desafio computacional clássico que envolve um tabuleiro quadrado de luzes, onde cada célula pode estar ativa (ligada) ou inativa (desligada). A mecânica do jogo determina que ao pressionar uma célula, seu estado é invertido juntamente com o estado de todas as células adjacentes nas quatro direções cardinais. O objetivo é desativar todas as luzes do tabuleiro.

Este problema apresenta características que o tornam particularmente interessante para análise sob a perspectiva da inteligência artificial: possui um espaço de estados finito, mas de crescimento exponencial conforme aumenta o tamanho do tabuleiro; envolve restrições complexas devido à interdependência das ações; e permite a aplicação de diferentes paradigmas de busca e otimização.

1.1 Objetivos

O presente estudo tem como objetivo principal avaliar comparativamente o desempenho de seis algoritmos distintos de busca e otimização quando aplicados ao problema Lights Out. Especificamente, busca-se:

1. Implementar uma plataforma experimental controlada para execução e análise de algoritmos;
2. Mensurar objetivamente o desempenho de cada estratégia através de métricas quantitativas;
3. Identificar as características que tornam determinados algoritmos mais adequados ao problema;
4. Analisar o comportamento dos algoritmos em tabuleiros de diferentes dimensões e complexidades.

1.2 Justificativa

A escolha do problema Lights Out como domínio de teste justifica-se por sua capacidade de evidenciar claramente as diferenças entre estratégias de busca informada e não informada, além de permitir a análise do comportamento de métodos de otimização local. A natureza discreta e determinística do problema facilita a reprodutibilidade dos experimentos e a interpretação dos resultados.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Espaço de Estados

O problema Lights Out pode ser modelado como um grafo de estados, onde cada configuração do tabuleiro representa um estado e cada ação (pressionar uma célula) representa uma aresta. Para um tabuleiro de dimensão $n \times n$, o espaço de estados possui $2^{(n^2)}$ configurações possíveis.

2.2 Algoritmos Implementados

2.2.1 Busca em Largura (BFS - Breadth Explorer)

A Busca em Largura explora o espaço de estados em níveis, garantindo que a primeira solução encontrada seja ótima em termos de número de movimentos. O algoritmo utiliza uma estrutura de fila para manter os estados a serem explorados e um conjunto de estados visitados para evitar reexplorações.

Características:

- Completude: Sim
- Otimalidade: Sim (menor número de movimentos)
- Complexidade de tempo: $O(b^d)$
- Complexidade de espaço: $O(b^d)$

2.2.2 Busca em Profundidade (DFS - Depth Explorer)

A Busca em Profundidade explora cada ramo do espaço de estados até sua máxima profundidade antes de retroceder. Para evitar explorações infinitas, implementou-se um limite de profundidade máxima e um timeout temporal.

Características:

- Completude: Não (com limitações de profundidade)
- Otimalidade: Não
- Complexidade de tempo: $O(b^m)$
- Complexidade de espaço: $O(bm)$

2.2.3 Busca Heurística A* (Heuristic Search)

O algoritmo A* combina informação sobre o custo acumulado com uma estimativa heurística do custo até o objetivo. No contexto do Lights Out, duas heurísticas foram consideradas:

5. Heurística de contagem: $h(n)$ = número de luzes ativas
6. Heurística de Manhattan: $h(n)$ = soma das distâncias mínimas até as bordas

A função de avaliação é: $f(n) = g(n) + h(n)$, onde $g(n)$ é o número de movimentos realizados.

Características:

- Completude: Sim
- Otimalidade: Sim (se heurística for admissível)
- Complexidade de tempo: $O(b^d)$
- Complexidade de espaço: $O(b^d)$

2.2.4 Busca Gulosa (Path Finder)

A Busca Gulosa utiliza apenas a função heurística para guiar a exploração, sempre escolhendo o estado que parece mais próximo do objetivo. Este método é mais rápido que o A*, mas não garante encontrar a solução ótima.

Características:

- Completude: Não
- Otimalidade: Não
- Complexidade de tempo: $O(b^m)$
- Complexidade de espaço: $O(b^m)$

2.2.5 Subida de Encosta (Hill Climber)

Algoritmo de otimização local que se move sempre para o vizinho com melhor avaliação heurística. Pode ficar preso em máximos locais.

Características:

- Completude: Não
- Otimalidade: Não
- Eficiência computacional: Alta
- Vulnerabilidade: Máximos locais, platôs

2.2.6 Têmpera Simulada (Annealing Optimizer)

Extensão probabilística da Subida de Encosta que aceita movimentos de piora com probabilidade decrescente, permitindo escapar de máximos locais.

Parâmetros implementados:

- Temperatura inicial: 100
- Taxa de resfriamento: 0,98
- Iterações por temperatura: 100

3 METODOLOGIA

3.1 Arquitetura do Sistema

O sistema desenvolvido foi estruturado em três módulos principais:

3.1.1 Módulo Core (Núcleo)

BoardGenerator (Gerador de Tabuleiros): Responsável pela criação de tabuleiros com níveis de dificuldade controlados. A dificuldade é determinada pelo número de movimentos aleatórios executados a partir do estado objetivo (todas as luzes apagadas), garantindo que o tabuleiro gerado possua solução.

GameState (Estado do Jogo): Classe que encapsula a representação de um estado do tabuleiro e fornece operações essenciais:

- `generate_neighbors()`: Gera todos os estados sucessores possíveis
- `check_victory()`: Verifica se o estado é objetivo
- `compute_active_count()`: Calcula o número de luzes ativas
- `compute_manhattan_metric()`: Calcula a heurística de Manhattan

3.1.2 Módulo Solver (Solucionadores)

Implementações dos seis algoritmos de busca, cada um em arquivo separado:

- `breadth_explorer.py`: Implementação BFS
- `depth_explorer.py`: Implementação DFS com limitações
- `heuristic_search.py`: Implementação A*
- `pathfinder.py`: Implementação Busca Gulosa
- `local_optimizer.py`: Implementações Hill Climbing e Simulated Annealing

3.1.3 Módulo de Análise

IsolatedSolverAnalyzer (batch_analyzer.py): Sistema automatizado de testes que:

7. Gera conjunto de tabuleiros com complexidade crescente
8. Executa cada algoritmo nos tabuleiros gerados
9. Coleta métricas de desempenho

10. Salva resultados em formato JSON estruturado
11. Trata interrupções do usuário preservando dados coletados

ExecutionManager (execution_manager.py): Coordena execuções em lote e gerencia timeouts.

3.2 Protocolo Experimental

Os experimentos foram conduzidos seguindo protocolo rigoroso:

1. Geração de Tabuleiros: Para cada tamanho (5×5 , 10×10 , 100×100), foram gerados 10 tabuleiros com dificuldade crescente, variando de 3 a 12 movimentos necessários para resolver.

2. Execução dos Algoritmos: Cada algoritmo foi executado isoladamente em todos os tabuleiros gerados.

3. Coleta de Métricas:

- Tempo de execução (segundos)
- Número de movimentos na solução
- Quantidade de estados explorados
- Taxa de sucesso (solução encontrada ou não)

4. Limites de Segurança:

- Timeout de 10 segundos para DFS
- Timeout de 30 segundos para algoritmos de otimização local
- Limite de profundidade de 50 níveis para DFS

5. Armazenamento: Todos os resultados foram salvos em arquivos JSON na pasta test_results/ com timestamp e configuração de teste.

3.3 Ambiente de Execução

- Linguagem: Python 3.x
- Estruturas de Dados: Collections.deque (BFS), heapq (A*, Gulosa)
- Sistema Operacional: Windows
- Período de Testes: Abril de 2026

4 RESULTADOS E DISCUSSÃO

4.1 Experimentos em Tabuleiros 5×5

Os tabuleiros 5×5 foram escolhidos como baseline para análise inicial, permitindo execução rápida e visualização clara das diferenças entre algoritmos.

4.1.1 Busca Heurística A*

Arquivo de resultado: isolated_Astar_5x5_20260413_204846.json

Métrica	Valor
Taxa de Sucesso	100% (10/10)
Movimentos (média)	6,9
Movimentos (mínimo)	3
Movimentos (máximo)	10
Tempo (média)	1,08 segundos
Tempo (mínimo)	0,0022 segundos
Tempo (máximo)	5,29 segundos
Estados explorados (média)	1.141,3

Análise: O A* demonstrou desempenho excepcional, resolvendo todos os casos de teste. A variação no tempo de execução (0,0022s a 5,29s) correlaciona-se diretamente com a complexidade do tabuleiro e o número de estados que precisam ser explorados. O algoritmo encontrou soluções ótimas em todos os casos.

4.1.2 Busca em Largura (BFS)

Status: Testes interrompidos devido a tempo excessivo de execução.

Arquivo: isolated_BFS_5x5_20260413_213125_interrupted.json

Análise: A BFS, apesar de garantir otimalidade, mostrou-se impraticável para uso em tabuleiros maiores devido ao crescimento exponencial do número de estados a serem mantidos em memória. Os testes foram interrompidos após tempo de execução excessivo.

4.1.3 Busca em Profundidade (DFS)

Arquivo de resultado: isolated_DFS_5x5_20260413_210339.json

Métrica	Valor
Taxa de Sucesso	0% (0/10)
Tempo médio	10,00 segundos (timeout)
Estados explorados (média)	15.314

Análise: A DFS falhou em todos os casos dentro do limite de tempo de 10 segundos. O algoritmo explorou em média 15.314 estados sem encontrar solução, evidenciando o problema de percorrer ramos profundos não promissores. A ausência de heurística e a natureza do espaço de estados tornam a DFS inadequada para este problema.

4.1.4 Busca Gulosa (Greedy Search)

Arquivo de resultado: isolated_Greedy_Search_5x5_20260413_204807.json

Métrica	Valor
Taxa de Sucesso	100% (10/10)
Movimentos (média)	15,3
Movimentos (mínimo)	3
Movimentos (máximo)	38
Tempo (média)	0,30 segundos
Tempo (mínimo)	0,0022 segundos
Tempo (máximo)	1,19 segundos
Estados explorados (média)	353,6

Análise: A Busca Gulosa conseguiu resolver todos os casos, porém com soluções subótimas. A média de 15,3 movimentos é significativamente superior aos 6,9 movimentos do A*, demonstrando o custo da ausência do componente $g(n)$ na função de avaliação. Entretanto, o

tempo médio de execução (0,30s) foi inferior ao do A* (1,08s), mostrando o trade-off entre otimalidade e velocidade.

4.1.5 Subida de Encosta (Hill Climbing)

Arquivo de resultado: isolated_Hill_Climbing_5x5_20260413_205756.json

Métrica	Valor
Taxa de Sucesso	50% (5/10)
Movimentos (média)	3,4 (apenas sucessos)
Movimentos (mínimo)	1
Movimentos (máximo)	5
Tempo (média)	0,0027 segundos
Estados explorados (média)	4,0

Análise: O Hill Climbing apresentou taxa de sucesso de apenas 50%, confirmando a vulnerabilidade a máximos locais. Nos casos em que encontrou solução, o tempo foi extremamente baixo (média de 0,0027s) e com poucas explorações (média de 4 estados). Isso demonstra que quando o algoritmo se inicia em uma região favorável do espaço de busca, converge rapidamente. Porém, sua falta de completude o torna inadequado para aplicações que exigem garantia de solução.

4.1.6 Têmpera Simulada (Simulated Annealing)

Arquivo de resultado: isolated_Simulated_Annealing_5x5_20260413_205349.json

Métrica	Valor
Taxa de Sucesso	10% (1/10)
Movimentos	19.641 (único sucesso)
Tempo (média)	27,87 segundos
Tempo (único sucesso)	16,15 segundos
Estados explorados (média)	41.623,9

Análise: Contrariando expectativas, a Têmpera Simulada apresentou desempenho inferior ao Hill Climbing, com apenas 10% de taxa de sucesso. O único caso resolvido requereu 19.641 movimentos, evidenciando que o algoritmo percorreu um caminho extremamente ineficiente. O tempo médio de 27,87 segundos e a alta quantidade de estados explorados sugerem que os parâmetros de temperatura e resfriamento podem não estar adequadamente calibrados para este problema específico.

4.2 Experimentos em Tabuleiros 10×10

Os tabuleiros 10×10 aumentam significativamente a complexidade, com espaço de estados de $2^{100} \approx 1,27 \times 10^{30}$ configurações.

4.2.1 Busca Heurística A*

Arquivo de resultado: isolated_Astar_10x10_20260413_213259.json

Métrica	Valor
Taxa de Sucesso	100% (10/10)
Movimentos (média)	12,8
Tempo (média)	142,5 segundos

Análise: O A* manteve 100% de taxa de sucesso, mas com aumento substancial no tempo de execução (142,5s em média), demonstrando o impacto do crescimento exponencial do espaço de estados.

4.2.2 Busca Gulosa

Arquivo de resultado: isolated_Greedy_Search_10x10_20260413_213238.json

Métrica	Valor
Taxa de Sucesso	100% (10/10)
Movimentos (média)	28,9
Tempo (média)	8,73 segundos

Análise: A Busca Gulosa manteve 100% de sucesso com tempo significativamente menor que o A* (8,73s vs 142,5s), mas com soluções de qualidade inferior (28,9 movimentos vs 12,8).

4.2.3 Subida de Encosta

Arquivo de resultado: isolated_Hill_Climbing_10x10_20260413_215508.json

Métrica	Valor
Taxa de Sucesso	20% (2/10)
Tempo (média)	0,015 segundos

Análise: A taxa de sucesso caiu para 20% em tabuleiros maiores, evidenciando que a probabilidade de máximos locais aumenta com a dimensão do problema.

4.3 Experimentos em Tabuleiros 100×100

Os tabuleiros 100×100 representam o teste extremo de escalabilidade.

4.3.1 Busca Heurística A*

Arquivos:

- isolated_Astar_100x100_20260415_210914.json
- isolated_Astar_100x100_20260415_211153.json

Métrica	Valor
Taxa de Sucesso	Variável
Tempo	Horas de processamento

Análise: Apenas os algoritmos heurísticos conseguiram abordar tabuleiros desta dimensão. O A* manteve capacidade de resolução, porém com tempos de execução da ordem de horas.

4.3.2 Busca Gulosa

Arquivo de resultado: isolated_Greedy_Search_100x100_20260415_211412.json

Análise: A Busca Gulosa mostrou-se mais escalável que o A* para tabuleiros muito grandes, conseguindo encontrar soluções (ainda que subótimas) em tempo razoável.

4.4 Análise Comparativa Global

Tabela Resumo - Tabuleiros 5×5

Algoritmo	Taxa de Sucesso	Tempo Médio	Movimentos Médios	Otimidade
A*	100%	1,08s	6,9	Sim
BFS	N/A	Timeout	N/A	Sim
DFS	0%	10,0s	N/A	N/A
Busca Gulosa	100%	0,30s	15,3	Não
Hill Climbing	50%	0,0027s	3,4*	Não
Simulated Annealing	10%	27,87s	19.641*	Não

*Apenas casos de sucesso

4.5 Discussão

Os resultados evidenciam claramente a superioridade dos algoritmos de busca informada (A* e Busca Gulosa) sobre métodos não informados e de otimização local no contexto do problema Lights Out.

Principais Observações:

- 1. Completude vs. Eficiência:** O A* apresenta o melhor equilíbrio, garantindo solução ótima com desempenho aceitável em tabuleiros pequenos e médios.
- 2. Trade-off Qualidade-Velocidade:** A Busca Gulosa oferece alternativa viável quando a otimalidade não é crítica e a velocidade é prioritária.
- 3. Inadequação de Busca Cega:** DFS e BFS mostraram-se impraticáveis devido a limitações de tempo e memória.

4. Limitações de Otimização Local: Hill Climbing e Simulated Annealing não são recomendados para este problema, apresentando baixa confiabilidade.

5. Escalabilidade: A dimensão do tabuleiro afeta drasticamente todos os algoritmos, mas métodos heurísticos degradam de forma mais controlada.

5 CONCLUSÃO

O presente estudo demonstrou empiricamente que algoritmos de busca informada são significativamente mais adequados para resolver o problema Lights Out quando comparados a métodos de busca cega ou otimização local. O A* destacou-se como a escolha ideal quando otimalidade é requerida, enquanto a Busca Gulosa apresenta-se como alternativa eficiente para cenários onde soluções aproximadas são aceitáveis.

Os resultados obtidos corroboram a teoria de que a qualidade da heurística é fator determinante no desempenho de algoritmos de busca em espaços de estados complexos. A heurística baseada na contagem de luzes ativas mostrou-se admissível e efetiva para guiar a exploração.

A metodologia desenvolvida, incluindo o sistema automatizado de testes e análise, provou ser robusta e extensível, podendo ser aplicada a outros domínios de problemas de busca. Os dados coletados fornecem base sólida para futuras investigações, incluindo:

- Exploração de heurísticas alternativas
- Técnicas de aprendizado de máquina para otimização de parâmetros
- Paralelização de algoritmos de busca
- Aplicação de métodos híbridos

5.1 Contribuições

Este trabalho contribui com:

- Implementação completa e documentada de seis algoritmos de busca aplicados ao Lights Out
- Framework experimental automatizado para análise comparativa
- Conjunto extenso de dados experimentais com métricas quantitativas
- Análise crítica das características que tornam algoritmos adequados ao problema

5.2 Limitações

Reconhece-se como limitações:

- Testes limitados a tabuleiros quadrados
- Parâmetros de Simulated Annealing não foram otimizados sistematicamente

- Análise de consumo de memória não foi detalhada
- Não foram exploradas variantes dos algoritmos básicos

5.3 Trabalhos Futuros

Recomenda-se para investigações futuras:

- Implementação de IDA* (Iterative Deepening A*) para reduzir consumo de memória
- Desenvolvimento de heurísticas baseadas em padrões (pattern databases)
- Aplicação de técnicas de aprendizado por reforço
- Estudo de decomposição do problema em subproblemas independentes
- Análise de variantes do Lights Out com diferentes topologias de vizinhança

REFERÊNCIAS

RUSSELL, S.; NORVIG, P. Artificial Intelligence: A Modern Approach. 4. ed. Pearson, 2020.

CORMEN, T. H. et al. Introduction to Algorithms. 3. ed. MIT Press, 2009.

HART, P. E.; NILSSON, N. J.; RAPHAEL, B. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. IEEE Transactions on Systems Science and Cybernetics, v. 4, n. 2, p. 100-107, 1968.

KIRKPATRICK, S.; GELATT, C. D.; VECCHI, M. P. Optimization by Simulated Annealing. Science, v. 220, n. 4598, p. 671-680, 1983.

GOLDREICH, O. Finding the Shortest Move-Sequence in the Graph-Generalized 15-Puzzle Is NP-Hard. Studies in Complexity and Cryptography, Springer, 2011.

ANDERSON, M.; CULLER, D. Lights Out! On a Set of Graphs. American Mathematical Monthly, v. 115, n. 10, p. 883-894, 2008.

APÊNDICE A - Estrutura de Dados dos Resultados

Exemplo de estrutura JSON gerada pelo sistema:

```
{
  "timestamp": "2026-04-13T20:48:46.318669",
  "config": {
    "algorithm": "A*",
    "num_tests": 10,
    "board_size": 5,
    "initial_difficulty": 3
  },
  "statistics": {
    "successes": 10,
    "failures": 0,
    "min_moves": 3,
    "avg_moves": 6.9,
    "max_moves": 10,
    "min_time": 0.002,
    "avg_time": 1.076,
    "max_time": 5.294
  },
  "details": [...]
}
```

APÊNDICE B - Instruções de Execução

Para reproduzir os experimentos:

```
from batch_analyzer import IsolatedSolverAnalyzer

# Exemplo: Teste A* em grid 5x5
analyzer = IsolatedSolverAnalyzer(
    'Heuristic Search',
    test_count=10,
    grid_size=5,
    base_difficulty=3
)

analyzer.run_complete_analysis()
```

APÊNDICE C - Diagrama da Arquitetura do Sistema

